Ambiguity and Uncertainty

08.01.2026

Contd...

Type of Ambiguity	Description
Lexical Ambiguity	• The ambiguity of a single word is called lexical ambiguity. • For example, treating the word **silver** as a noun, an adjective, or a verb.
Syntactic Ambiguity	• This kind of ambiguity occurs when a sentence is parsed in different ways. • For example, the sentence "The man saw the girl with the telescope". It is ambiguous whether the **man saw the girl carrying a telescope** or **he saw her through his telescope.**

Contd...

Type of Ambiguity	Description
Semantic Ambiguity	▪ This kind of ambiguity occurs when the meaning of the words themselves can be misinterpreted. ▪ In other words, semantic ambiguity happens when a sentence contains an ambiguous word or phrase. ▪ For example, the sentence "**The car hit the pole while it was moving**" is having semantic ambiguity because the interpretations can be "**The car, while moving, hit the pole**" and "**The car hit the pole while the pole was moving**".

Contd...

Type of Ambiguity	Description
Anaphoric Ambiguity	• This kind of ambiguity arises due to the use of anaphora entities in discourse. • For example, the horse ran up the hill. It was very steep. • It soon got tired. Here, the anaphoric reference of **"it"** in two situations cause ambiguity.
Pragmatic ambiguity	• Such kind of ambiguity refers to the situation where the context of a phrase gives it multiple interpretations. • In simple words, we can say that pragmatic ambiguity arises when the statement is not specific. • For example, the sentence **"I like you too"** can have multiple interpretations like I like you (just like you like me), I like you (just like someone else dose).

NLP Functions

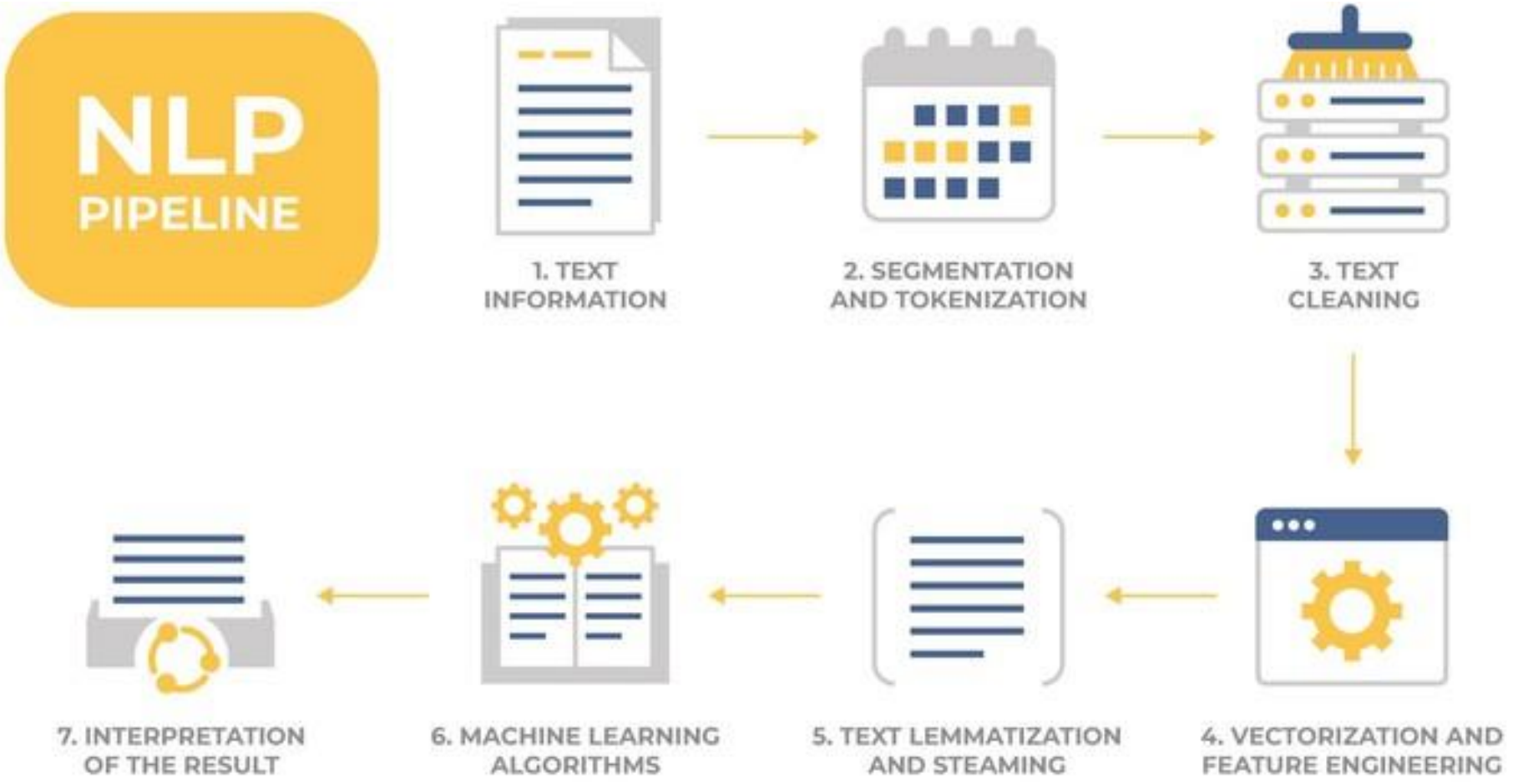
Among the tasks that Natural Language Processing solves the most important ones are:

- **Machine translation:** is the first classic task assigned to the developers of NLP-technologies (it is necessary to note that it is not yet solved at the necessary level of quality for today).
- **Grammar and spell checking** – as a conclusion of the first task.
- **Text classification** – definitions of text semantics for further processing (one of the most popular tasks to date).

Contd...

- **Summarization –** the text generalization to a simplified version form (re-interpretation the content of the texts).
- **Text generation –** one of the tasks that are used to build AI-systems
- **Topic modeling –** technique for extracting hidden topics from large text volumes.

Contd...


NLP
PIPELINE
1. TEXT INFORMATION
2. SEGMENTATION AND TOKENIZATION
3. TEXT CLEANING
4. VECTORIZATION AND FEATURE ENGINEERING
5. TEXT LEMMATIZATION AND STEAMING
6. MACHINE LEARNING ALGORITHMS
7. INTERPRETATION OF THE RESULT

Contd...

- Enter the text (or sound converted to text)
- **Segmentation of text** into components (segmentation and tokenization).
- **Text Cleaning** (filtering from "garbage") – removal of unnecessary elements.
- **Text Vectorization** and Feature engineering.
- **Lemmatization and Steaming** – reducing inflections for words.
- Using Machine Learning algorithms and methods for training models.
- Interpretation of the result

Metrices used in NLP

Metric	Function
Edit Distance	One of the simple and at the same time popularly usable metrics is Edit distance (sometimes is known as *Levenshtein distance*) – an algorithm for estimating the similarity of two string values (word, words form, words composition), by comparing the minimum number of operations to convert one value into another.

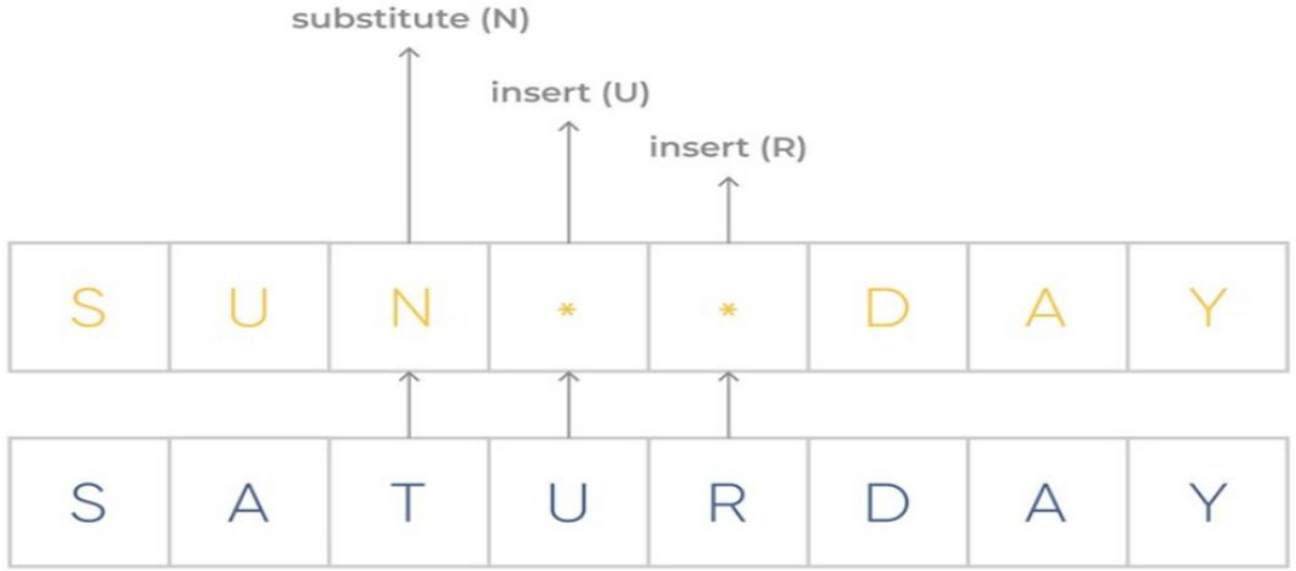
An example of Edit distance execution is shown below.

```
One operation - Edit distance: 1
str1 = "string", str2 = "strong"

Several operations - Edit distance: 3
str1 = "sunday", str2 = "saturday"
```

So this algorithm includes the following text operations:

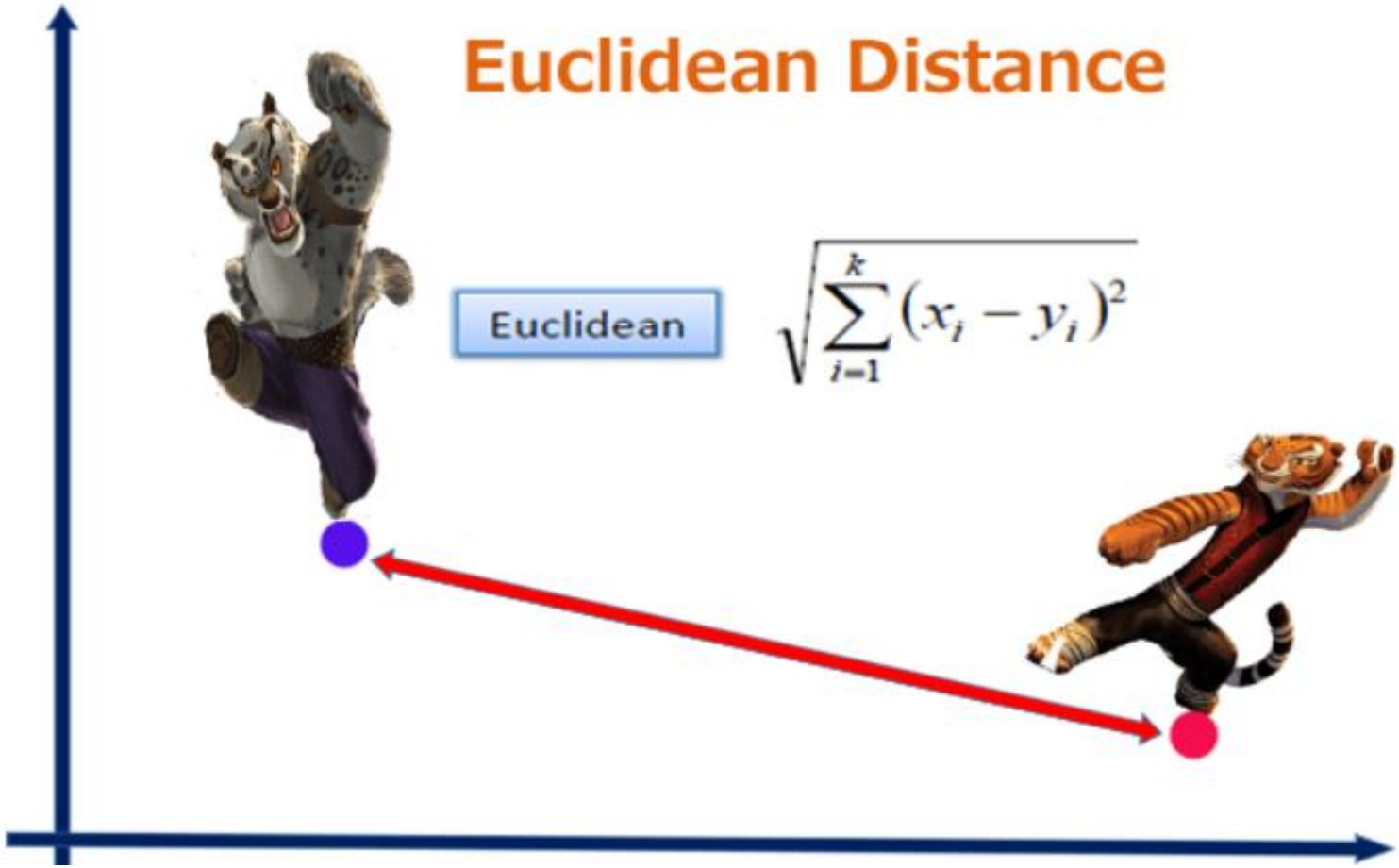
- inserting a character into a string;
- delete (or replace) a character from a string by another character;
- characters substitutions.



Metrices used in NLP

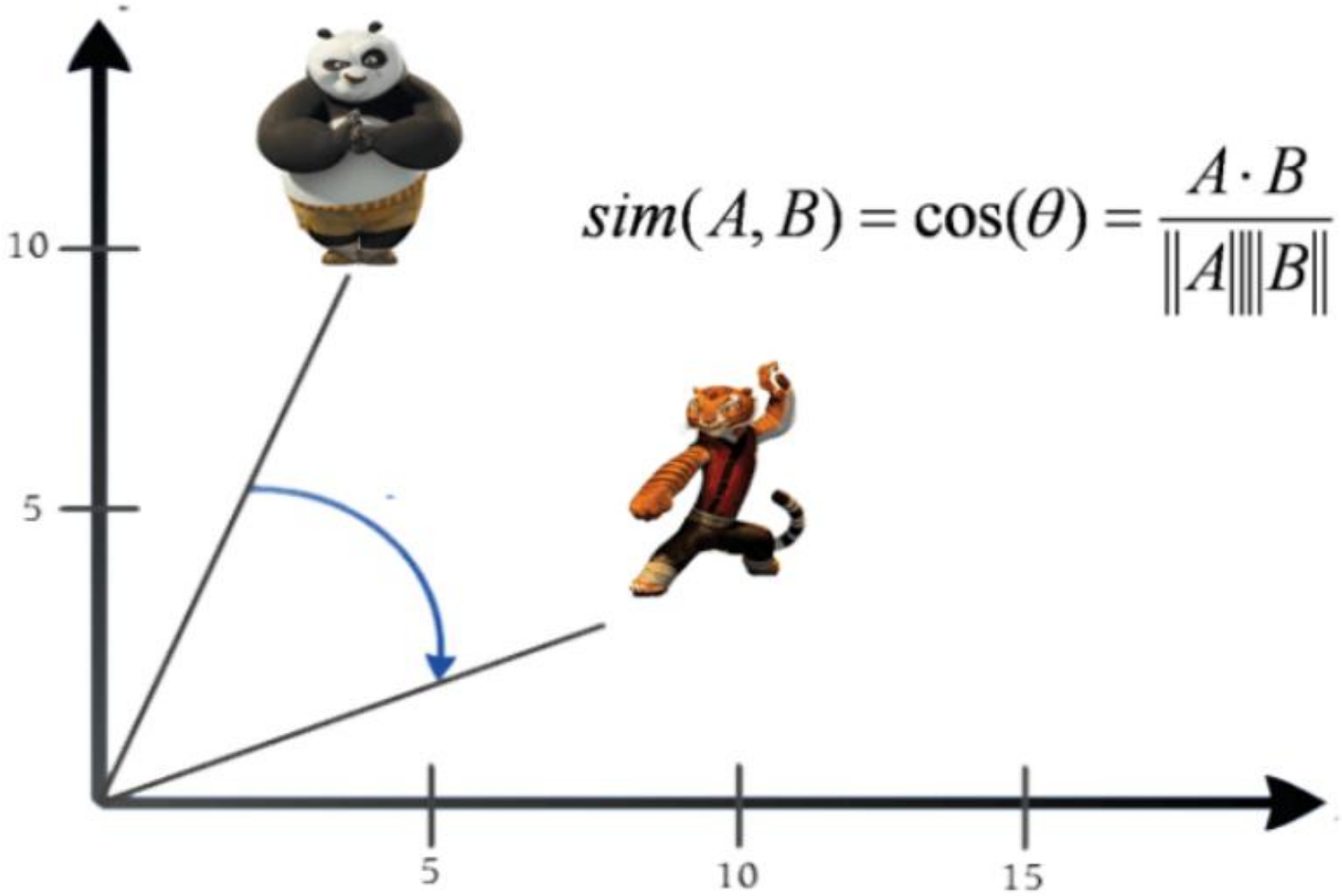
Metric	Function
Euclidean Distance	Euclidean distance (often called L2 norm) is the most intuitive of the metrics. Comparing the shortest distance among two objects. Score means the distance between two objects. If it is 0, it means that both objects are identical.

Euclidean Distance



Metrices used in NLP

Metric	Function
Cosine Similarity	Determine the angle between two objects is the calculation method to the find similarity. The range of score is 0 to 1. If score is 1, it means that they are same in orientation (not magnitude).



$$sim(A,B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

Metrices used in NLP

Metric	Function
Jaccard Index	The measurement is refer to number of common words over all words. More commons mean both objects should be similarity.

Jaccard Similarity = (Intersection of A and B) / (Union of A and B)

The range is 0 to 1. If score is 1, it means that they are identical. There is no any common word between the first sentence and the last sentence so the score is 0. The following example shows score when comparing the first sentence.

$|A| = 4$ $|B| = 5$

MODELS AND ALGORITHMS

Models and Algorithms

The most important model:

- State machines
- Formal rule systems
- Logic
- Probability theory
- other machine learning tools

The most important algorithms of these models:

- State space search algorithms
- Dynamic programming algorithm

Contd...

- State machines are formal models that consist of states, transitions among states, and an input representation.

Some of the variations of this basic model:

- Deterministic and non-deterministic finite-state automata
- finite-state transducers - which can write to an output device
- weighted automata, Markov models, and hidden Markov models - which have a probabilistic component.

Contd...

- Closely related to the above procedural models are their declarative counterparts: formal rule systems.
- Regular grammars and Regular relations, context-free grammars, feature-augmented grammars, as well as probabilistic variants of them all.
- State machines and formal rule systems are the main tools used when dealing with knowledge of phonology, morphology, and syntax.

Contd...

- Representative tasks include searching through a space of phonological sequences for a likely input word in speech recognition, or searching through a space of trees for the correct syntactic parse of an input sentence.
- Among the algorithms that are often used for these tasks are well-known graph algorithms such as **depth-first search**, as well as heuristic variants such as **best-first**, and **A* search**.

Contd...

- The third model that plays a critical role in capturing knowledge of language is **logic**.
- The first order logic, also known as the **predicate calculus**, as well as such related formalisms as feature-structures, semantic networks, and conceptual dependency.
- These logical representations have traditionally been the tool of choice when dealing with knowledge of semantics, pragmatics, and discourse (applications in these areas are increasingly relying on the simpler mechanisms used in phonology, morphology, and syntax).

Contd...

- One major use of probability theory is to solve the many kinds of ambiguity problems;
- almost any speech and language processing problem can be recast as: “given N choices for some ambiguous input, choose the most probable one”.

Another major advantage of probabilistic models is that they are one of a class of machine learning models.

- Machine learning research has focused on ways to automatically learn the various representations automata, rule systems, search heuristics, classifiers.

Popular models in NLP

Keyword Extraction

- Keywords Extraction is one of the most important tasks in Natural Language Processing, and it is responsible for determining various methods for extracting a significant number of words and phrases from a collection of texts.
- All of this is done to summarize and assist in the relevant and well-organized organization, storage, search, and retrieval of content.
- There are numerous keyword extraction algorithms available, each of which employs a unique set of fundamental and theoretical methods to this type of problem.

Contd...

- There are various types of NLP algorithms, some of which extract only words and others which extract both words and phrases.
- There are also NLP algorithms that extract keywords based on the complete content of the texts.

Contd...

The following are some of the most prominent keyword extraction algorithms:

- **TextRank:** This algorithm operates on the same idea as PageRank. Google uses this method to rank the importance of various websites on the internet.

- **Term Frequency –** Inverse Document Frequency (TF-IDF): The full version of TF-IDF is Term Frequency – Inverse Document Frequency, which tries to better define the importance of a term in a document. Also, take into account the relationships between texts from the same corpus.

- **RAKE:** RAKE stands for Rapid Automatic Keywords Extraction and is a type of NLP method. This can extract keywords and key phrases from a single document's content without taking into account other documents in the same collection.

Knowledge Graphs

- A knowledge graph is a way of storing data that resulted from an information extraction task.
- Many basic implementations of knowledge graphs make use of a concept we call triple, that is a set of three items(a subject, a predicate and an object) that we can use to store information about something.

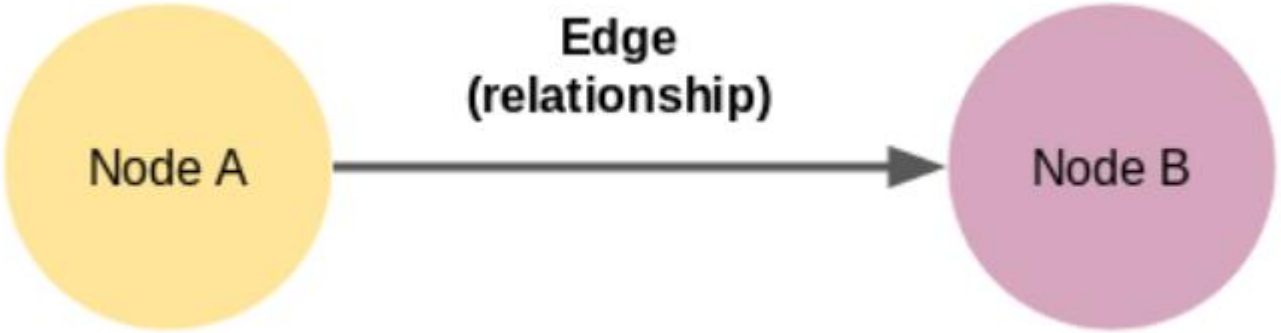
Data Representation in Knowledge Graph

London is the capital of England. Westminster is located in London.

- After some basic processing which we will see later, we would 2 triples like this:

(London, be capital, England), (Westminster, locate, London)

Contd...

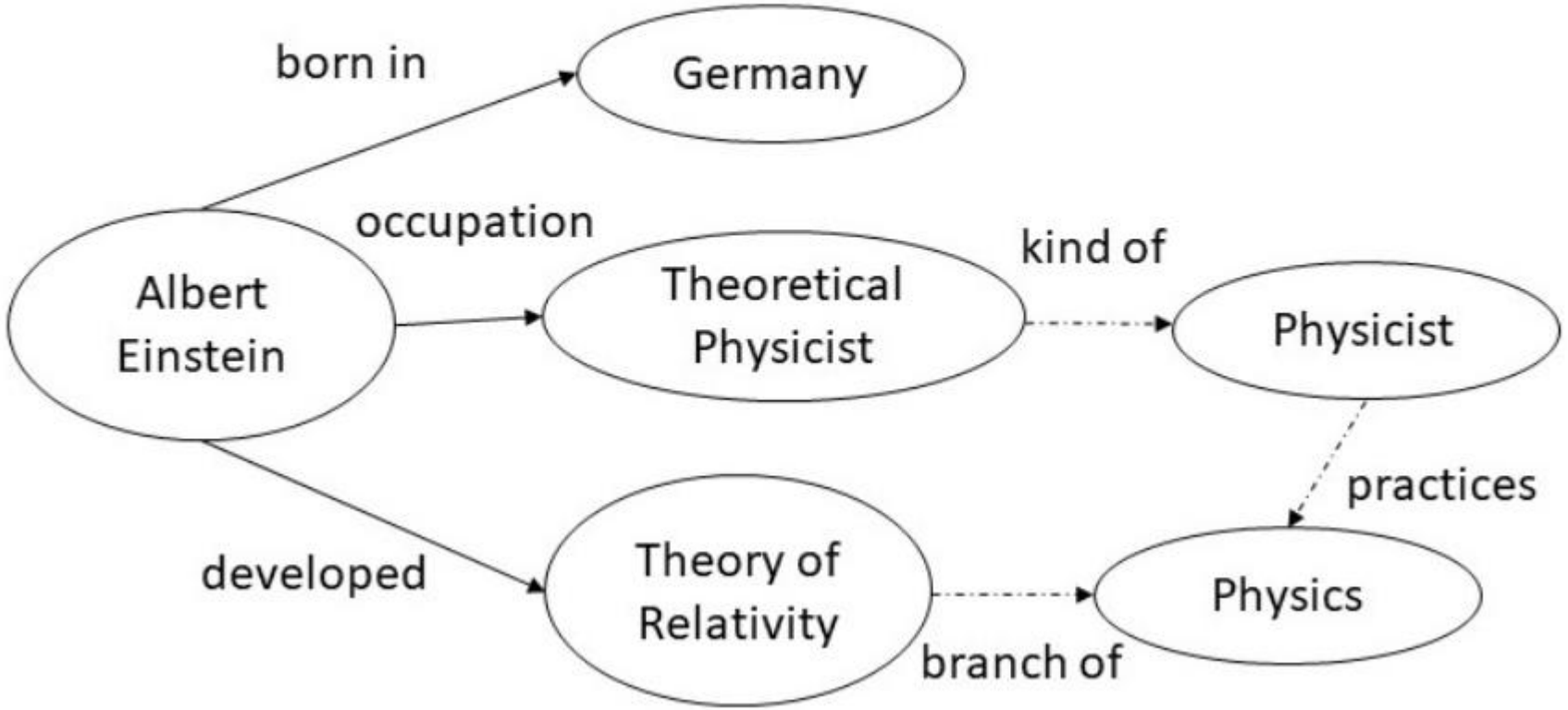


Node A and Node B here are two different entities. These nodes are connected by an edge that represents the relationship between the two nodes. Now, this is the smallest knowledge graph we can build – it is also known as a triple.

Contd...

- Three unique entities(London, England and Westminster) and two relations (be capital, locate).
- To build a knowledge graph, we only have two associated nodes in the graph with the entities and vertices with the relations.
- To build a knowledge graph from the text, it is important to make our machine understand natural language.
- Using NLP techniques such as sentence segmentation, dependency parsing, parts of speech tagging, and entity recognition.

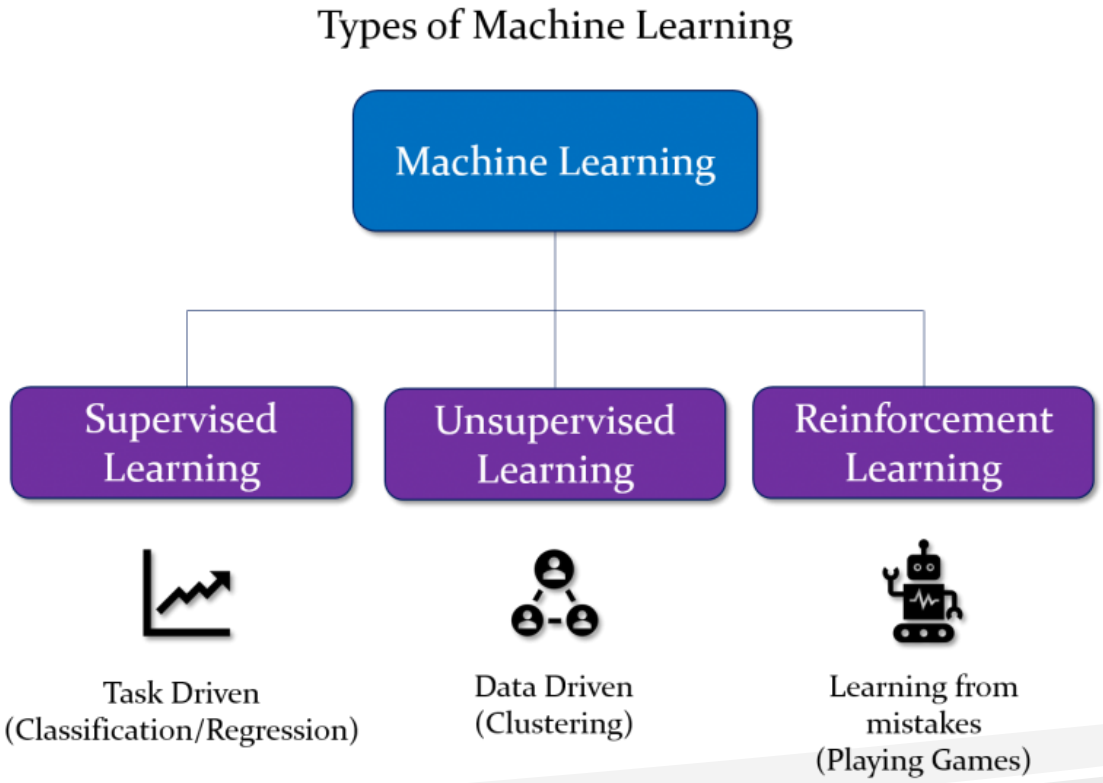
Albert Einstein was a **German-born theoretical physicist** who developed the **theory of relativity**.

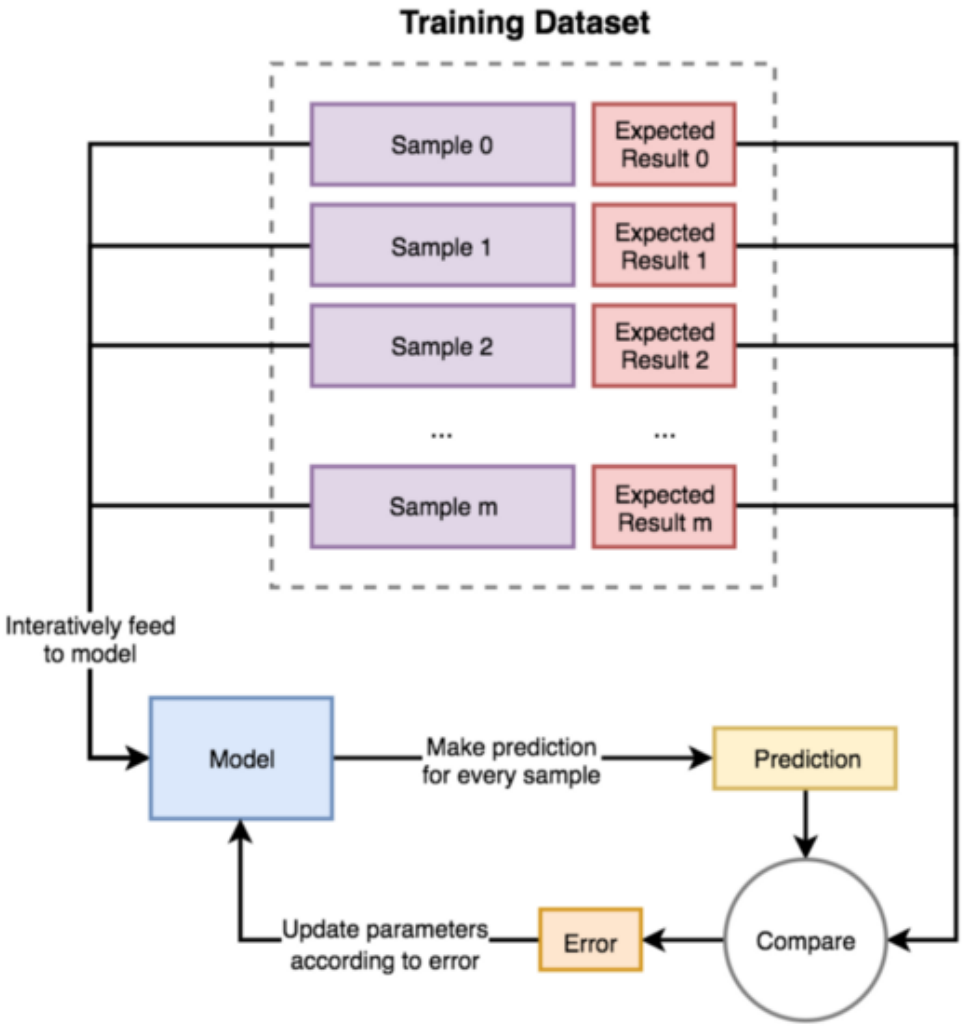


Learning models in NLP

Within artificial intelligence (AI) and machine learning, there are two basic approaches: supervised learning and unsupervised learning.

The main difference is one uses labeled data to help predict outcomes, while the other does not.



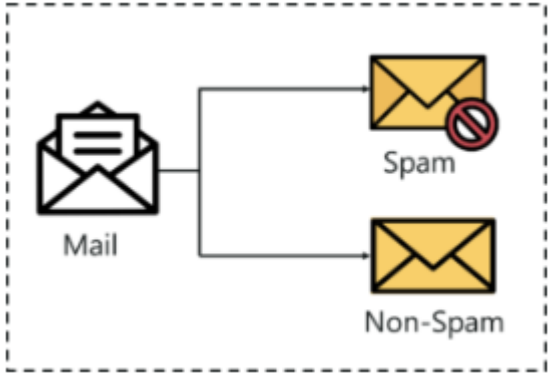

Training Dataset
Sample 0
Expected Result 0
Sample 1
Expected Result 1
Sample 2
Expected Result 2
...
...
Sample m
Expected Result m
Interatively feed to model
Model
Make prediction for every sample
Prediction
Compare
Error
Update parameters according to error

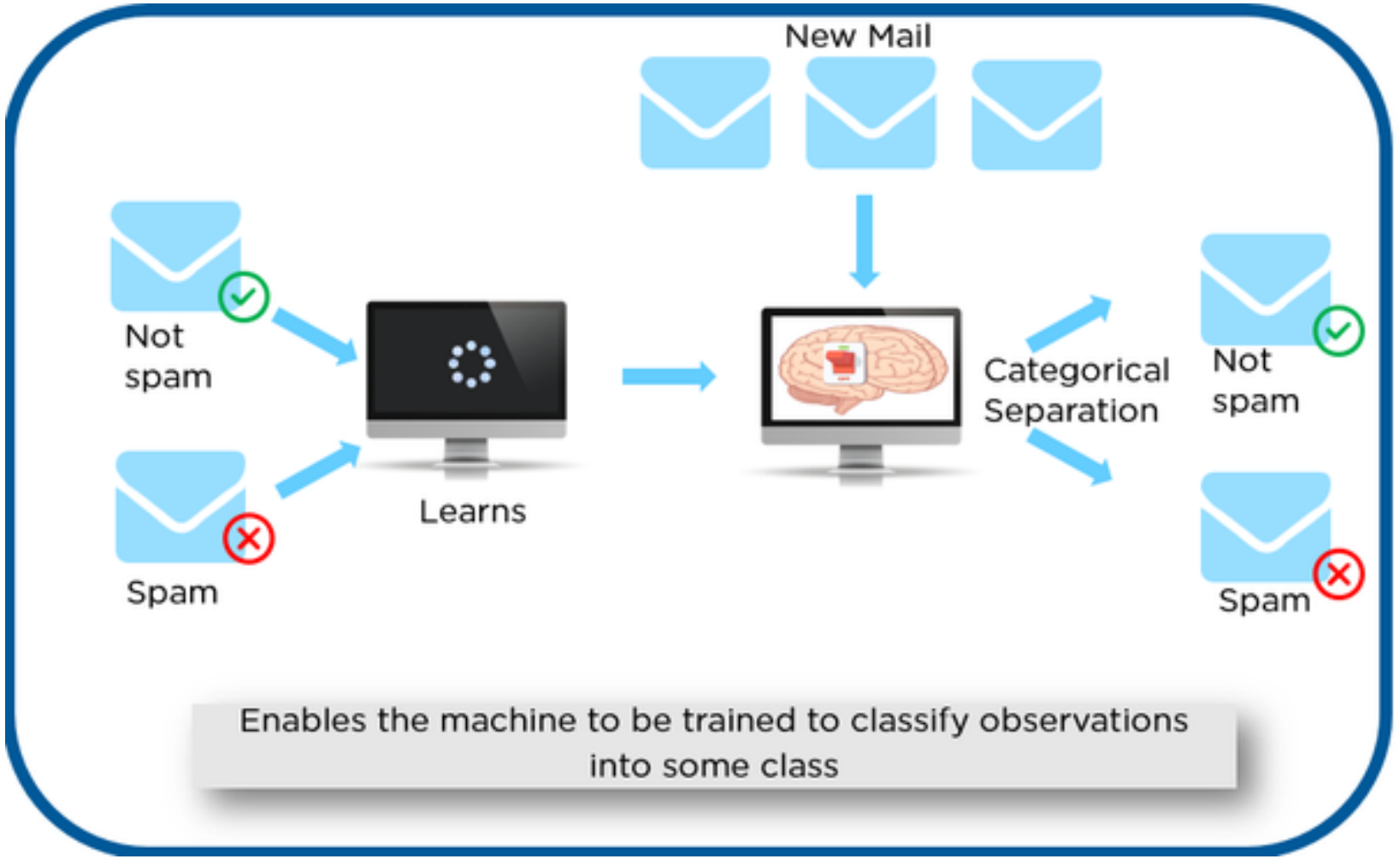
Types of Supervised learning

- **Classification**: process of *assigning category to input* data sample.
  - Example usages: predicting whether a person is ill or not, detecting fraudulent transactions, face classifier.
- **Regression**: process of *predicting a continuous, numerical value* for input data sample.
  - Example usages: assessing the house price, forecasting grocery store food demand, temperature forecasting.

Classification in NLP

- Classification is a process of categorizing a given set of data into classes, It can be performed on both structured or unstructured data.
- The process starts with predicting the class of given data points. The classes are often referred to as target, label or categories.
- The classification predictive modeling is the task of approximating the mapping function from input variables to discrete output variables.




New Mail
Not
spam
Spam
Learns
Categorical
Separation
Not
spam
Spam
Enables the machine to be trained to classify observations into some class

Contd...

- The main goal is to identify which class/category the new data will fall into.

Simple example:

- Heart disease detection can be identified as a classification problem, this is a binary classification since there can be only two classes i.e has heart disease or does not have heart disease. The classifier, in this case, needs training data to understand how the given input variables are related to the class. And once the classifier is trained accurately, it can be used to detect whether heart disease is there or not for a particular patient.

Language, Thought, and Understanding

Contd...

- The effective use of language is intertwined with our general cognitive abilities.
- **Turing Test** by Alan Turing (1950)
- He suggested an empirical test, a game, in which a computer's use of language would form the basis for determining if it could think.
- If the machine could win the game it would be judged intelligent.
- **ELIZA** was an early natural language processing system capable of carrying on a limited form of conversation with a user.

Contd...

- Consider the following session with a version of ELIZA that imitated the responses of a Rogerian psychotherapist.

User1: You are like my father in some ways.
ELIZA1: What Resemblance Do You See
User2: You are not very aggressive but I think you don't want me to notice that.
ELIZA2: What Makes You Think I Am Not Aggressive
User3: You don't argue with me.
ELIZA3: Why Do You Think I Don't Argue With You
User4: You are afraid of me.
ELIZA4: Does It Please You To Believe I Am Afraid Of You

Contd...

- ELIZA is a remarkably simple program that makes use of pattern-matching to process the input and translate it into suitable outputs.
- The success of this simple technique in this domain is due to the fact that ELIZA doesn't actually need to know anything to mimic a Rogerian psychotherapist.

The State of the Art and the Near-term Future

Contd...

- The rise of the World-Wide Web, have placed speech and language processing applications in the spotlight, and have pointed out a plethora of exciting possible applications.
- The following scenarios serve to illustrate some current applications and near-term possibilities
- Eg: A **Canadian computer** program accepts **daily weather data** and generates weather reports that are passed along unedited to the public in **English and French.**

Contd...

- **Eg 2:** A **visitor to Cambridge, Massachusetts**, asks a computer about places to eat using only spoken language.
- The system returns relevant information from a database of facts about the local restaurant scene.
- **Eg 3:** A computer reads hundreds of typed **student essays and assigns grades** to them in a manner that is indistinguishable from human graders.

Contd...

- A **satellite operator** uses language to ask questions and commands to a computer that controls a **world-wide network of satellites** (?).
- **German and Japanese entrepreneurs** negotiate a time and place to meet in their own languages using **small hand-held communication devices** (?).
- Closed-captioning is provided in in any of a number of languages for a broadcast news program by a computer listening to the audio signal (?).
- A **computer equipped with a vision system** watches a professional soccer game and provides an **automated natural language account of the game** (?).

Regular Expression

Regular Expression

- **Regular expression**, the standard notation for characterizing text sequences.
- The regular expression is used for specifying text strings in situations like this web-search example, and in other information retrieval applications, but also plays an important role in word-processing (in PC, Mac, or UNIX applications), computation of frequencies from corpora, and other such tasks.

Regular expressions

- A formal language for specifying text strings
- How can we search for any of these?
  - woodchuck
  - woodchucks
  - Woodchuck
  - Woodchucks

- For example, to search for woodchuck - **/woodchuck/.**

Contd…

- The simplest kind of regular expression is a sequence of simple characters.
- So the regular expression /Buttercup/ matches any string containing the substring Buttercup, for example the line **I'm called little Buttercup.**

Contd…

RE	Example Patterns Matched
/woodchucks/	“interesting links to woodchucks and lemurs”
/a/	“Mary Ann stopped by Mona’s”
/Claire says,/	“Dagmar, my gift please,” Claire says,”
/song/	“all our pretty songs”
/!/	“You’ve left the burglar behind again!” said Nori

Contd…

Regular expressions are case sensitive; **lower-case /s/** is distinct from **upper-case /S/;** (/s/ matches a lower case s but not an upper-case S)

RE	Match	Example Patterns
/[wW]oodchuck/	Woodchuck or woodchuck	"Woodchuck"
/[abc]/	'a', 'b', or 'c'	"In uomini, in soldati"
/[1234567890]/	any digit	"plenty of 7 to 5"

Contd…

In these cases the brackets can be used with the dash (-) to specify any one character in a range. The pattern /[2- 5]/ specifies any one of the characters 2, 3, 4, or 5. The pattern /[b-g]/ specifies one of the characters b, c, d, e, f, or g. Some other examples:

RE	Match	Example Patterns
/[A-Z]/	an uppercase letter	"we should call it 'Drenched Blossoms'"
/[a-z]/	a lowercase letter	"my beans were impatient to be hoed!"
/[0-9]/	a single digit	"Chapter 1: Down the Rabbit Hole"

Contd…

RE	Match (single characters)	Example Patterns
[^A-Z]	not an uppercase letter	“Oyfn pripetchik”
[^Ss]	neither ‘S’ nor ‘s’	“I have no exquisite reason for’t”
[^\.]	not a period	“our resident Djinn”
[e^]	either ‘e’ or ‘^’	“look up ^ now”
a^b	the pattern ‘a^b’	“look up a^ b now”

Contd…

The question-mark ? marks optionality of the previous expression

RE	Match	Example Patterns
woodchucks?	woodchuck or woodchucks	“Woodchuck”
colou?r	color or colour	“colour”

Contd...

- Consider the language of (certain) sheep, which consists of strings that look like the

  baa!

  baaa!

  baaaa!

  baaaaa!

  baaaaaa!

  . . .

- The set of operators that allow us to say things like “some number of ‘a’s” are based on the asterisk or *, commonly called the Kleene * (pronounced “cleany star”).
- The Kleene star means KLEENE * ‘zero or more occurrences of the immediately previous character or regular expression’

/a*/ - ‘any string of zero or more a’s’

- This will match **a or aaaaaa** but it will also **match Off Minor**, since the string Off Minor has **zero a’s**.
- So the regular expression for matching **one or more a is /aa*/,** meaning one **a** followed by **zero or more a’s**

Contd...

- **/[ab]*/** means ‘zero or more ‘a’s or ‘b’s’
- Match a String **aaa, ababab, or bbbb**
- The regular expression for an individual digit was **/[0-9]/**
- The regular expression for an integer (a string of digits) is

  /[0-9][0-9]*/
- The expression **/[0-9]+/** is the normal way to specify ‘a sequence of digits’
- Two ways to specify the sheep language: **/baaa*!/ or /baa+!/**

Contd...

- One very important special character is the period **/./**

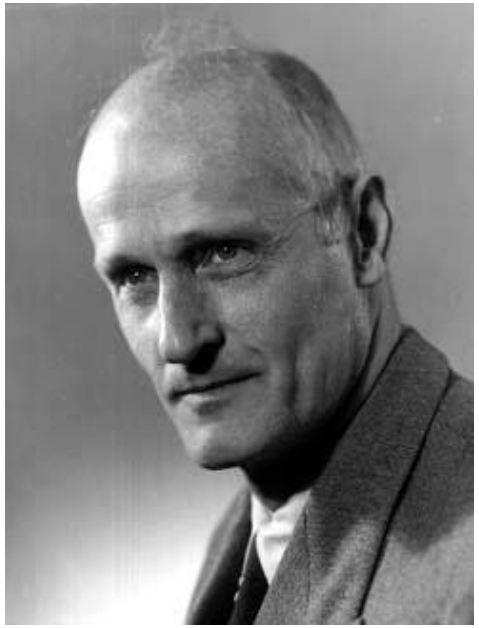
RE	Match	Example Patterns
/beg.n/	any character between ‘beg’ and ‘n’	begin, beg’n, begun

To find any line in which a particular word, for example **aardvark**, appears twice.

The regular expression **/aardvark.*aardvark/**

Regular Expressions: ? * + .

Pattern	Matches	
colou?r	Optional previous char	color colour
oo*h!	0 or more of previous char	oh! ooh! oooh! ooooh!
o+h!	1 or more of previous char	oh! ooh! oooh! ooooh!
baa+		baa baaa baaaa baaaaa
beg.n		begin begun begun beg3n



Stephen C Kleene

Kleene *, Kleene +

Contd...

- **Anchors** are special characters that anchor regular expressions to particular places in a string.
- The most common anchors are the caret ^ and the dollar-sign **$**.
- The caret ^ matches **the start of a line**. The pattern **/^The/** matches the word **The only at the start of a line**.
- Thus there are **three uses of the caret ^**:
  - to match the start of a line
  - as a negation inside of square brackets
  - just to mean a caret

Contd...

- The dollar sign **$** matches the **end of a line**.
- The pattern **$** is a useful pattern for matching a space at the end of a line, and **/^The dog\.$/** matches a **line that contains only the phrase The dog.**
- **\b** matches a word boundary,
- **\B** matches a non-boundary.
- Thus **/\b the\b/** matches the word ***the*** but not the word ***other***

Contd...

- **Perl** defines a word as any sequence of digits, underscores or letters;
- For example, /\b99/ will match the string 99 in **"There are 99 bottles of cooldrinks on the wall"** (because 99 follows a space) but not 99 in
- **There are 299 bottles of cooldrinks on the wall** (since 99 follows a number).
- But it will match 99 in $99 (since 99 follows a dollar sign ($), which is not a digit, underscore, or letter)

Disjunction, Grouping, and Precedence

- The disjunction operator is used to search for text, also called the pipe symbol **|** .
- The pattern **/cat|dog/** matches either the string **cat** or the string **dog**.
- If - Both guppy and guppies? To search in a sentence

/guppy|ies/ - cannot be used

/gupp(y|ies)/ - Enclosing a pattern in parentheses

Contd...

- A line that has column labels of the form **Column 1 Column 2 Column 3.**
- The expression **/Column [0-9]+ */** will not match any column; instead, it will match a column followed by any number of spaces!
- The **star** here applies only to the **space that precedes it**, not the whole sequence.
- With the parentheses, we could write the expression

/(Column [0-9]+ *)*/ - to match the word Column, followed by a number and optional spaces, the whole pattern repeated any number of times.

operator precedence hierarchy for RE

Higher to lower precedence	
Parenthesis	( )
Counters	* + ? {}
Sequences and anchors	the ^my end$
Disjunction	|

Example

- **/the*/** - **theeeeee** but not **thethe**

  counters have a higher precedence than **sequences**

- **/the|any/** - **the or any** but not **theny**

  sequences have a higher precedence than **disjunction**

A simple example

- write a RE to find cases of the English article **the**

  /the/

- This pattern will miss the word when it begins a sentence and hence is capitalized

  /[tT]he/

- Incorrectly return texts with the embedded in other words (e.g. other or theology).
- Instances with a word boundary on both sides:

  /\b[tT]he\b/

Contd...

- It won't treat **underscores and numbers as word boundaries**; like (**the_ or the25**)

/[^a-z][tT]he[^a-z]/

- one more problem with this pattern: it won't find the word the when it begins a line.
- This is because the regular expression [^az], which we used to avoid embedded **thes**, implies that there must be some single (although non-alphabetic) character before the **the**. We can avoid this by specifying that before the **the** we require either the beginning-of-line or a non-alphabetic character: **/(^|[^a-z])[tT]he[^a-z]/**

Advanced Operators

RE	Expansion	Match	Example Patterns
\d	[0-9]	any digit	Party␣of␣5
\D	[^0-9]	any non-digit	Blue␣moon
\w	[a-zA-Z0-9␣]	any alphanumeric or space	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[␣\r\t\n\f]	whitespace (space, tab)	
\S	[^\s]	Non-whitespace	in␣Concord

Regular expression operators for counting

RE	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	exactly zero or one occurrence of the previous char or expression
{n}	n occurrences of the previous char or expression
{n,m}	from n to m occurrences of the previous char or expression
{n,}	at least n occurrences of the previous char or expression

Some characters that need to be backslashed

RE	Match	Example Patterns Matched
*	an asterisk "*"	"K*A*P*L*A*N"
\.	a period "."	"Dr. Livingston, I presume"
\?	a question mark	"Would you light my candle?"
\n	a newline	
\t	a tab	

Activity

25.01.2024

Example

Python code uses regular expressions to search for the word in the given string and then prints the start and end indices of the matched word within the string.

Code

```
import re

s = ‘Natural Language Processing is a course deals with natural languages'

match = re.search(r’deals', s)

print('Start Index:', match.start())

print('End Index:', match.end())
```

Output:

- Start Index: 40
- End Index: 45

Function	Description
re.findall()	finds and returns all matching occurrences in a list
re.compile()	Regular expressions are compiled into pattern objects
re.split()	Split string by the occurrences of a character or a pattern.
re.sub()	Replaces all occurrences of a character or patter with a replacement string.
re.escape()	Escapes special character
re.search()	Searches for first occurrence of character or pattern.

Example - Finding all occurrences of a pattern

\d+ - To find all the sequences of one or more digits in the given string. It searches for numeric values and stores them in a list.

```
['e', 'a', 'd', 'b', 'e', 'a']
```

Code

```
import re

string = """“My pincode number of postal address is 430117
and my mobile number is 8827242475"""

regex = '\d+'

match = re.findall(regex, string)

print(match)
```

Output:

- [‘430117', ' 8827242475 ']

Example - re.compile()

The code uses a regular expression pattern[a-e] to find and list all lowercase letters from ‘a’ to ‘e’ in the input string “Bob, said Mr. Aldren soloman to go for a walk”.

Code

```
import re
p = re.compile('[a-e]')
print(p.findall('Bob, said Mr. Aldren soloman to go for a walk'))
```

Output:

- ['b', 'a', 'd', 'd', 'e', 'a', 'a', 'a']

Example - class [\s,.] will match any whitespace character, ‘,’ or, ‘.’ .

The code uses regular expressions to find and list all single digits and sequences of digits in the given input strings. It finds single digits with **\d** and sequences of digits with **\d+**

Code

```
import re
p = re.compile('\d')
print(p.findall("I visited to college at 9 A.M. on 10th June 2022"))
p = re.compile('\d+')
print(p.findall("I visited to college at 9 A.M. on 10th June 2022 "))
```

Output:

- [‘9’, ‘1', ‘0', ‘2', ‘0’, ‘2’, ‘2']
- [‘9', ‘10', ‘2022']

Example

The code uses regular expressions to find and list word characters, sequences of word characters, and non-word characters in input strings. It provides lists of the matched characters or sequences.

Code

```
import re
p = re.compile('\w')
print(p.findall("He said * in some_lang."))
p = re.compile('\w+')
print(p.findall("I went to him at 11 A.M., he \
said *** in some_language."))
p = re.compile('\W')
print(p.findall("he said *** in some_language."))
```

Output:

- ['H', 'e', 's', 'a', 'i', 'd', 'i', 'n', 's', 'o', 'm', 'e', '_', 'l', 'a', 'n', 'g']
- ['I', 'went', 'to', 'him', 'at', 'ıı', 'A', 'M', 'he', 'said', 'in', 'some_language']
- [' ', ' ', '*', '*', '*', ' ', ' ', '.']

Example

The code uses a regular expression pattern ‘ba*’ to find and list all occurrences of ‘ba’ followed by zero or more ‘a’ characters in the input string “babaabbbaaa”.

Code

```
import re
p = re.compile(‘ba*')
print(p.findall("babaabbbaaa"))
```

Output:

- [‘ba', ‘baa', ‘b’, ’b’, ‘baaa']

re.split()

- Split string by the occurrences of a character or a pattern, upon finding that pattern, the remaining characters from the string are returned as part of the resulting list.

Syntax:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

Contd...

- The First **parameter**, pattern denotes the regular expression
- **string** is the given string in which pattern will be searched for and in which splitting occurs
- **maxsplit** if not provided is considered to be zero '0', and if any nonzero value is provided, then at most that many splits occur. If maxsplit = 1, then the string will split once only, resulting in a list of length 2.
- The **flags** are very useful and can help to shorten code, they are not necessary parameters, eg: flags = re.IGNORECASE, in this split, the case, i.e. the lowercase or the uppercase will be ignored.

Code

```
from re import split
print(split('\W+', ‘Names, names , Names'))
print(split('\W+’, “Name’s names Names"))
print(split('\W+', ‘On 25th Jan 2025, at 02:50 PM'))
print(split('\d+', On 25th Jan 2025, at 02:50 PM '))
```

Output:

- ['H', 'e', 's', 'a', 'i', 'd', 'i', 'n', 's', 'o', 'm', 'e', '_', 'l', 'a', 'n', 'g']
- ['I', 'went', 'to', 'him', 'at', 'ıı', 'A', 'M', 'he', 'said', 'in', 'some_language']
- [' ', ' ', '*', '*', '*', ' ', ' ', '.']

re.sub()

- The ‘sub’ in the function stands for SubString, a certain regular expression pattern is searched in the given string(3rd parameter), and upon finding the substring pattern is replaced by repl(2nd parameter), count checks and maintains the number of times this occurs.

Syntax:

```
re.sub(pattern, repl, count=0, flags=0)
```

Code

```
import re
print(re.sub('ub', '~*', 'Subject has Uber booked already',
             flags=re.IGNORECASE))
print(re.sub('ub', '~*', 'Subject has Uber booked already'))
print(re.sub('ub', '~*', 'Subject has Uber booked already',
             count=1, flags=re.IGNORECASE))
print(re.sub(r'\sAND\s', ' & ', 'Baked Beans And Spam',
             flags=re.IGNORECASE))
```

Output:

- S~*ject has ~*er booked already
- S~*ject has Uber booked already
- S~*ject has Uber booked already
- Baked Beans & Spam

re.escape()

- Returns string with all non-alphanumerics back slashed, this is useful if you want to match an arbitrary literal string that may have regular expression metacharacters in it.

Syntax:

re.escape(string)

Code

```
import re

print(re.escape("This is Awesome even 1 AM"))

print(re.escape("I Asked what is this [a-9], he said \t ^WoW"))
```

Output:

- This\ is\ Awesome\ even\ 1\ AM
- I\ Asked\ what\ is\ this\ \[a\-9\]\,\ he\ said\ \	\ \^WoW

re.search()

- This method either returns None (if the pattern doesn't match), or a re.MatchObject contains information about the matching part of the string. This method stops after the first match, so this is best suited for testing a regular expression more than extracting data.

Code

```
import re
regex = r"([a-zA-Z]+) (\d+)"
match = re.search(regex, "I was born on June 24")
if match != None:
    print ("Match at index %s, %s" % (match.start(), match.end()))
    print ("Full match: %s" % (match.group(0)))
    print ("Month: %s" % (match.group(1)))
    print ("Day: %s" % (match.group(2)))
else:
    print ("The regex pattern does not match.")
```

Output:

- Match at index 14, 21
- Full match: June 24
- Month: June
- Day: 24

Character	Description
\	Used to drop the special meaning of character following it
[]	Represent a character class
^	Matches the beginning
$	Matches the end
.	Matches any character except newline
|	Means OR (Matches with any of the characters separated by it.)

Example

The first search (re.search(r'.', s)) matches any character, not just the period, while the second search (re.search(r'\.', s)) specifically looks for and matches the period character

Code

```
import re

s = ‘start .learning NLP'

# without using \

match = re.search(r'.', s)

print(match)

# using \

match = re.search(r'\.', s)

print(match)
```

Example - Square Brackets ([]) represent a character class consisting of a set of characters that we wish to match

```
import re

string = “Write a pattern which places the first word of an English sentence in a register"

pattern = "[a-m]"

result = re.findall(pattern, string)

print(result)
```

Caret (^) symbol matches the beginning of the string i.e. checks whether the string starts with the given character(s) or not

```
import re

regex = r'^The'

strings = ['The quick brown fox', 'The lazy dog', 'A quick brown fox']

for string in strings:
    if re.match(regex, string):
        print(f'Matched: {string}')
    else:
        print(f'Not matched: {string}')
```

Output:

Matched: The quick brown fox

Matched: The lazy dog

Not matched: A quick brown fox

Dollar($) symbol matches the end of the string i.e checks whether the string ends with the given character(s) or not

```
import re
string = "Hello World!"
pattern = r"World!$"
match = re.search(pattern, string)
if match:
    print("Match found!")
else:
    print("Match not found.")
```

```
import re
test_string = '123abc456789abc123ABC'
pattern = re.compile(r"abc")
matches = pattern.finditer(test_string)

for match in matches:
    print(match)
```

Output:

```
<re.Match object; span=(3, 6), match='abc'>
<re.Match object; span=(12, 15), match='abc'>
```